

Recursion and lambda function

Recursion:

Recursion is a programming technique where a function calls itself to solve a smaller instance of the same problem until it reaches a base condition (stopping point).

Why Use Recursion?

To break complex problems into simpler subproblems.

Useful for tasks like factorial, Fibonacci series, tree/graph traversal, tower of Hanoi, etc.

Structure of a Recursive Function

1. Every recursive function has:
2. Base case – Stops recursion.

Recursive call – Calls itself with a smaller input.

```
In [1]: def wish():  
        print('hello')  
        print('hi')  
        wish() # this not a recursion
```

```
hello  
hi
```

```
In [4]: def wish():  
        print('Hello')  
        wish()  
        wish() # not recursion
```

```
Hello  
Hello
```

```
In [9]: import sys  
        sys.setrecursionlimit(150)  
        print(sys.getrecursionlimit())  
  
        i = 0  
        def wish():  
            global i  
            i += 1  
            print('Hello', i)  
            wish()
```

```
wish()
```

```
# i set recursion limit 150 but it will print only 125 times
```

```
150
Hello 1
Hello 2
Hello 3
Hello 4
Hello 5
Hello 6
Hello 7
Hello 8
Hello 9
Hello 10
Hello 11
Hello 12
Hello 13
Hello 14
Hello 15
Hello 16
Hello 17
Hello 18
Hello 19
Hello 20
Hello 21
Hello 22
Hello 23
Hello 24
Hello 25
Hello 26
Hello 27
Hello 28
Hello 29
Hello 30
Hello 31
Hello 32
Hello 33
Hello 34
Hello 35
Hello 36
Hello 37
Hello 38
Hello 39
Hello 40
Hello 41
Hello 42
Hello 43
Hello 44
Hello 45
Hello 46
Hello 47
Hello 48
Hello 49
Hello 50
Hello 51
Hello 52
Hello 53
Hello 54
Hello 55
```

Hello 56
Hello 57
Hello 58
Hello 59
Hello 60
Hello 61
Hello 62
Hello 63
Hello 64
Hello 65
Hello 66
Hello 67
Hello 68
Hello 69
Hello 70
Hello 71
Hello 72
Hello 73
Hello 74
Hello 75
Hello 76
Hello 77
Hello 78
Hello 79
Hello 80
Hello 81
Hello 82
Hello 83
Hello 84
Hello 85
Hello 86
Hello 87
Hello 88
Hello 89
Hello 90
Hello 91
Hello 92
Hello 93
Hello 94
Hello 95
Hello 96
Hello 97
Hello 98
Hello 99
Hello 100
Hello 101
Hello 102
Hello 103
Hello 104
Hello 105
Hello 106
Hello 107
Hello 108
Hello 109
Hello 110
Hello 111

```
Hello 112  
Hello 113  
Hello 114  
Hello 115  
Hello 116  
Hello 117  
Hello 118  
Hello 119  
Hello 120  
Hello 121  
Hello 122  
Hello 123  
Hello 124  
Hello 125
```

```

-----
RecursionError                                Traceback (most recent call last)
Cell In[9], line 11
      9     print('Hello', i)
     10     wish()
--> 11 wish()

Cell In[9], line 10, in wish()
      8 i += 1
      9 print('Hello', i)
--> 10 wish()

Cell In[9], line 10, in wish()
      8 i += 1
      9 print('Hello', i)
--> 10 wish()

[... skipping similar frames: wish at line 10 (122 times)]

Cell In[9], line 10, in wish()
      8 i += 1
      9 print('Hello', i)
--> 10 wish()

Cell In[9], line 9, in wish()
      7 global i
      8 i += 1
----> 9 print('Hello', i)
     10 wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:664, in OutStream.write(self, string)
     655 def write(self, string: str) -> Optional[int]: # type:ignore[override]
     656     """Write to current stream after encoding if necessary
     657
     658     Returns
     659     (...)
     662
     663     """
--> 664     parent = self.parent_header
     666     if not isinstance(string, str):
     667         msg = f"write() argument must be str, not {type(string)}" # type:ig
nore[unreachable]

RecursionError: maximum recursion depth exceeded

```

```
In [10]: sys.getrecursionlimit()
```

```
Out[10]: 150
```

```
In [11]: sys.setrecursionlimit(1000)
         sys.getrecursionlimit()
```

```
Out[11]: 1000
```

Factorial using Recursion

```
In [12]: def fact(n):  
         if (n == 0 or n == 1): # base condition  
             return 1  
         return n * fact(n - 1)  
  
         result = fact(5)  
         print(result)
```

120

Fibonacci Series using Recursion

```
In [13]: def fibonacci(n):  
         if n <= 1: # Base case  
             return n  
         return fibonacci(n-1) + fibonacci(n-2)  
  
         print(fibonacci(6))
```

8

```
In [14]: def fibonacci(n):  
         if n <= 1:  
             return n  
         return fibonacci(n-1) + fibonacci(n-2)  
  
         def print_series(num):  
             for i in range(num):  
                 print(fibonacci(i), end=" ")  
  
         print_series(7)
```

0 1 1 2 3 5 8

Anonymous function | lambda function

```
In [16]: def square(a): # this is not Lambda function  
         return a * a # we can do this using lambda function  
         square(5)
```

Out[16]: 25

```
In [18]: f = lambda a : a * a # Lambda function single line function, it is very fast  
         square = f(5)  
         print(square)
```

25

```
In [20]: add = lambda a, b : a + b  
         sub = lambda a, b : a - b
```

```
subtract = sub(4,1)

print(add(4,1))
print(subtract)
```

5
3

```
In [21]: f = lambda a, b : a + b
        f1 = lambda a, b : a - b
        f2 = lambda a, b : a * b

        add = f(4, 1)
        sub = f1(4, 1)
        mul = f2(4, 2)

        print(add)
        print(sub)
        print(mul)
```

5
3
8

How can we use lambda in other function like - filter, map & reduce

filter()

```
In [23]: def is_even(n):
        return n % 2 == 0

        nums = [3,2,6,8,4,6,2,9]

        even = list(filter(is_even, nums))
        print(even)
```

[2, 6, 8, 4, 6, 2]

```
In [25]: def is_odd(n):
        return n % 2 != 0

        nums = [3,2,6,8,4,6,2,9]

        odd = list(filter(is_odd, nums))
        print(odd)
```

[3, 9]

```
In [26]: nums = [3,2,6,8,4,6,2,9]

        evens = list(filter(lambda n : n%2 == 0, nums))
        print(evens)
```


[2, 6, 8, 4, 6, 2]

```
In [27]: nums = [3,2,6,8,4,6,2,9]

odds = list(filter(lambda n : n%2 != 0, nums))
print(odds)
```

[3, 9]

```
In [28]: nums = [3,2,6,8,4,6,2,9, 34,77, 120]

even = list(filter(lambda n : n%2 == 0, nums))
odd = list(filter(lambda n : n%2 != 0, nums))

print(even)
print(odd)
```

[2, 6, 8, 4, 6, 2, 34, 120]

[3, 9, 77]

map()

- What ever even number I have from the assigned list
- I want to double the even number i.e 2 become 4 || 4 become 6 || 6 become 8
- that we will do using map function
- this largely we are using in google map reduce programme
- we can build using user define & lambda

```
In [30]: def update(n):
          return n+2
nums = [3,2,6,8,4,6,2,9]

evens = list(filter(lambda n : n%2 == 0, nums))
double = list(map(update, evens))

print(evens)
print(double)
```

[2, 6, 8, 4, 6, 2]

[4, 8, 10, 6, 8, 4]

```
In [31]: nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n: n*2, evens))

print(evens)
print(double)
```

[2, 6, 8, 4, 6, 2]

[4, 12, 16, 8, 12, 4]

```
In [33]: nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(lambda n : n%2 == 0, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n + 2, evens))
double_l = list(map(lambda n : n - 2, evens))

print(evens)

print(double)
print(double_)
print(double_l)
```

```
[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
[4, 8, 10, 6, 8, 4]
[0, 4, 6, 2, 4, 0]
```

reduce()

- i want to perform reduce now
- i want reduce all the values
- reduce you can add only 2 values
- [4, 12, 16, 8, 12, 4] if you sum everything then you will get 56

```
In [34]: from functools import reduce

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
sums = reduce(lambda a, b : a + b, double)

print(evens)
print(double)
print(sums)
```

```
[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
56
```

Python Decorators(@):

- Python decorator is nothing but a function which takes another function as argument and return a function
- it adds extra functionality to another function without modifying its actual code.

```
In [35]: def div(a, b): # this is not decorator
          print(a / b)
```

```
div(4, 2)
```

2.0

```
In [36]: def div(a, b):
          if a < b:
              a, b = b, a
          print(a / b)
          div(2, 4)
```

2.0

```
In [37]: # Lets write real code of decorator
```

```
In [38]: def div(a, b):
          print(a / b)

          def div_decorator(func):
              def inner(a, b):
                  if a < b:
                      a, b = b, a
                  return func(a, b)
              return inner

          div = div_decorator(div) # this is decorator
          div(2, 4)
```

2.0

```
In [42]: def my_decorator(func):
          def wrapper():
              print('Something is happening before the function is called')
              #func()
              print('Something is happening after the function is called')
          return wrapper

          @my_decorator
          def say_hello():
              print('Hello')

          say_hello()
```

Something is happening before the function is called
Something is happening after the function is called

```
In [41]: def my_decorator(func):
          def wrapper():
              print('Something is happening before the function is called')
              func()
              print('Something is happening after the function is called')
          return wrapper

          @my_decorator
          def say_hello():
              print('Hello')
```

```
say_hello()
```

Something is happening before the function is called

Hello

Something is happening after the function is called

```
In [43]: import time

def timer(func):
    def wrapper():
        start = time.time()
        func()
        end = time.time()
        print(f"Execution Time: {end - start} seconds")
    return wrapper

@timer
def test_function():
    for _ in range(1000000):
        pass

test_function()
```

Execution Time: 0.027459144592285156 seconds

```
In [44]: def repeat(n):
def decorator(func):
    def wrapper():
        for _ in range(n):
            func()
    return wrapper
return decorator

@repeat(3)
def greet():
    print("Hello!")

greet()
```

Hello!

Hello!

Hello!

```
In [ ]:
```